

RVD

Design notes for a Voronoi and restricted Voronoi engine you can look inside

30 September 2026

Status. Design only, no code yet. Written against `mesh-viewer` at `18b9619` and `mesh-dev` at `dc7151c`. Everything said about the viewer was checked against its source, and everything said about the mathematics has a reference at the end.

This document is meant to be read from start to finish once, then used for reference. §1 is the whole idea on one page. §3 is the mathematics, kept to what the design needs. Sections 4 to 7 are the system: its layers, its data, its predicates, its algorithms. §8 is how it sits on your viewer. §9 is the order to build it in, and §10 is how each step is tested. §11 answers the question you asked directly: how the plain Voronoi diagram becomes a restricted one, and why the design is built so that this step is small.

Contents

1	The idea on one page	4
2	What exists, and the gap	5
3	The mathematics the design rests on	6
3.1	Cells are intersections of half-spaces	6
3.2	Restriction is clipping, piece by piece	6
3.3	The dual: the restricted Delaunay triangulation	7
3.4	What is integrated, and why	7
3.5	Optimal transport: why the weights exist from day one	7
3.6	Degeneracy, stated honestly	8
4	Architecture	8
4.1	Layers, and which way they depend	8
4.2	What to reuse from mesh-dev	9
4.3	The stages and what each one produces	9
4.4	Stage B streams, and consumers listen	9
4.5	Freshness	10
4.6	Parallel, and bit-identical	10
5	Data structures	10
5.1	PointSet: the seeds	10
5.2	The spatial index: nearest seeds	11
5.3	Domain: where the diagram lives	11
5.4	Constraints: the planes that bound a clipped piece	11
5.5	Symbolic vertices: the heart of it	11
5.6	ClippedPolygon: a triangle clipped by one cell	13
5.7	ConvexCell: a box or tet clipped by one cell, stored as its dual	13

5.8	NeighbourGraph	13
5.9	Generation: one result, frozen	13
6	Predicates and robustness	14
6.1	The one rule: predicates read input numbers only	14
6.2	The predicate family	14
6.3	Fast when easy, exact when not	15
6.4	Ties: symbolic perturbation	16
6.5	The oracle: a second, stupid implementation	16
6.6	Counters and switches	16
6.7	Constructions: numbers for looking at, not for deciding	17
7	Algorithms	17
7.1	Stage A: which half-spaces bound each cell	17
7.1.1	Brute force (the reference)	17
7.1.2	Nearest seeds and a security radius (the default)	17
7.1.3	Delaunay (optional, later)	18
7.2	The 3D clipper	18
7.3	The polygon clipper	18
7.4	Stage B: which pieces each cell meets	19
7.5	Stage C: the consumers	19
7.6	Stage D: the optimisers	20
8	On top of mesh-viewer	21
8.1	The host, in the viewer's own terms	21
8.2	What is published	21
8.3	Overlay channels	22
8.4	Generations, ids, and keeping what the user set	22
8.5	The frame, and where the work happens	23
8.6	Windows	24
8.7	Picking a cell	24
8.8	Where the viewer pinches, and two small extensions	24
9	Implementation plan	25
M0.	The skeleton	25
M1.	Predicates	25
M2.	One cell, then all of them the slow way	25
M3.	Fast Voronoi in a box	25
M4.	Restricted to a surface	25
M5.	Integration and Lloyd	26
M6.	Restricted to a volume	26
M7.	Weights and transport	26
M8.	Speed and second opinions	26
M9.	Outward	26
10	Testing	26
10.1	The harness	26
10.2	Predicates against the oracle	27
10.3	Invariants, on everything	27
10.4	Known answers	27
10.5	Two ways, one answer	27
10.6	Optimisers	28
10.7	Against Geogram (optional)	28

10.8	Determinism	28
10.9	Benchmarks	28
10.10	The gallery	28
11	From a Voronoi diagram to a restricted one	29
11.1	The short version	29
11.2	The rules that keep the step small	29
11.3	Further, in order of effort	29
11.4	Where it could meet your own work	30
12	What makes this worth building	30
12.1	A robustness microscope	30
12.2	A closed-ball inspector	31
12.3	Cell replay, and “why does this vertex exist?”	31
12.4	Optimisation you can watch	31
12.5	Any cell, instantly	31
12.6	Results you can defend	31
13	Decisions for you, and risks	31
13.1	Decisions	31
13.2	Risks, and what contains them	32
14	Glossary	32

1 The idea on one page

You are building a C++17 library, and a mesh-viewer host program around it. The library computes the Voronoi diagram of a set of points, the *seeds*, optionally with a weight each (a *power* or *Laquerre* diagram). It computes it clipped to a box, restricted to a triangle surface, or restricted to the inside of a tetrahedral mesh. From the cells it produces a mesh to look at, the mass and centroid of every cell, the restricted Delaunay triangulation, and a list of everything that is wrong with the result. On top of that sit the optimisers that make Voronoi diagrams useful: Lloyd and L-BFGS for centroidal Voronoi tessellations, and Newton on the weights for semi-discrete optimal transport.

One sentence organises the whole design:

A Voronoi cell is a convex polytope cut out by half-spaces.
A restricted cell is those same half-spaces applied to the pieces of a domain.

So the engine is one *clipper*, fed by a *neighbour oracle* that says which half-spaces bound each cell, and feeding a set of *consumers* that turn clipped pieces into whatever you want (figure 1).

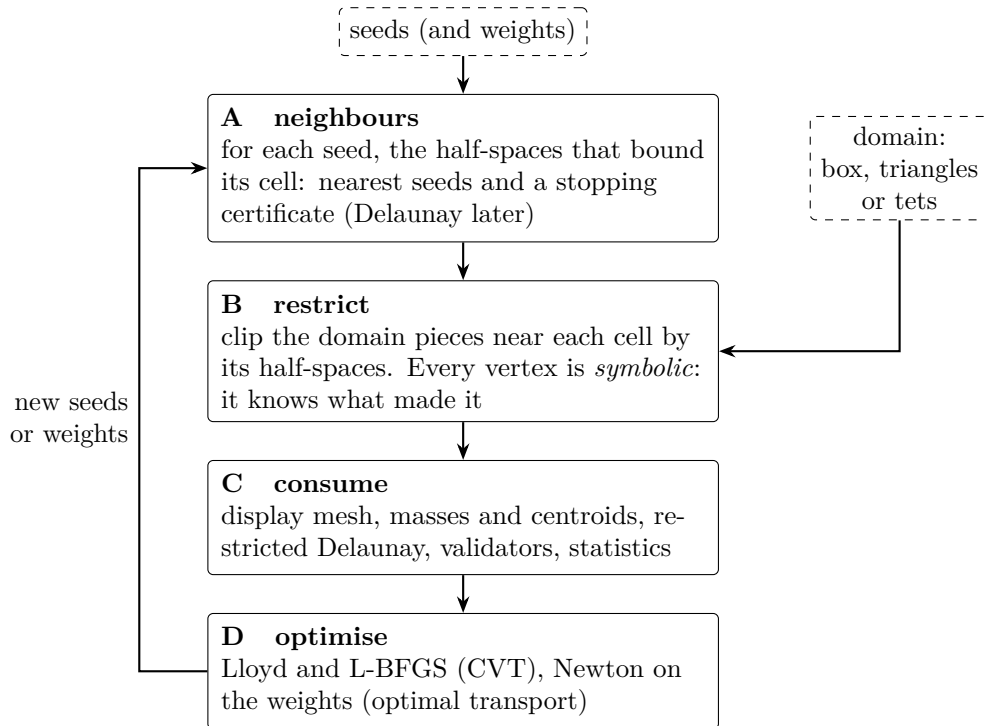


Figure 1: The four stages. A computes the Voronoi diagram in the domain’s bounding box, B restricts it to the domain, C turns the clipped pieces into results, and D moves the seeds or the weights and starts again.

Three decisions shape everything after this page.

1. **A Voronoi diagram in a box is a restricted Voronoi diagram**, whose domain happens to be one convex piece. So the Voronoi maker you build first is not a separate program that the RVD later replaces. It is stage A and stage B running on the simplest possible domain, and every line of it survives into the RVD.
2. **Every vertex is symbolic**. A vertex is stored as *what defines it* (“triangle 812, where the bisectors of seeds 4–17 and 4–23 cross it”), with its coordinates only as a derived, display-grade copy. Every decision the algorithm takes is a predicate evaluated exactly,

from the input numbers alone. That single choice buys exactness, vertex welding without tolerances, the dual complex for free, and a debugger that can say why any vertex exists.

3. **The viewer is a host concern.** The core never includes a GL header, exactly as in mesh-dev. A result is an immutable *generation* that the host publishes through `MeshSource` and `OverlaySource`. The viewer does not change for phase one. Two small, general viewer extensions are proposed for later, in §8.8, and neither is required.

What will make it more than another Voronoi code (§12): a *robustness microscope* that shows which predicates needed exact arithmetic and where the input was degenerate. A *closed-ball inspector* that shows exactly where your sampling is too coarse for the restricted Delaunay triangulation to be a valid mesh. *Cell replay*, to watch one cell being clipped bisector by bisector. Optimisation you can watch converge live. And results that are bit-identical on one thread and on sixty-four, tested against constants known in closed form.

2 What exists, and the gap

Your impression that there is no good RVD software online is fair, with one large exception.

software	what it does well	why it is not what you want
Geogram (B. Lévy, Inria)	The reference. Delaunay in 3D, restricted Voronoi diagrams on surfaces and volumes, power diagrams, exact predicates through its Predicate Construction Kit, CVT remeshing (<i>vorpalite</i>), and semi-discrete optimal transport	Dense code inside a large framework with its own mesh type, attribute system and conventions. Built to be fast and correct, not to be read, changed or looked inside: nothing in it shows you, on the mesh, why a given vertex exists or where exact arithmetic was needed. You already benchmark against it (<code>mesh-dev/tests/bench_vs_geogram.cpp</code>), and it should stay your second opinion here too (§10.7)
CGAL	Delaunay and regular triangulations in 3D with exact predicates, and mesh generation that uses restricted Delaunay triangulations internally	As far as I know it has no public restricted Voronoi diagram of a triangle mesh for arbitrary seeds: no cells restricted to a surface, no cell integrals, no CVT on a surface
voro++ (Rycroft, 2009)	Fast, well-documented 3D Voronoi cells, one at a time, in boxes and simple walls	Floating-point tolerances, and no triangle-mesh or tet-mesh domains. It does radical (power) diagrams, but has no CVT or transport on top
Qhull, scipy.spatial.Voronoi	Convex hulls, Delaunay triangulations, unbounded Voronoi diagrams	No clipping and no restriction. Unbounded cells come back with a vertex “at infinity”
GPU research codes (Ray et al., 2018; Basselin et al., 2021)	Very fast Voronoi and restricted power diagrams, “meshless”, one cell per thread	Floating point, and written for their papers’ experiments rather than as libraries. Their <i>algorithm</i> , though, is the one this design adopts for stage A (§7.1.2)

Geogram is at <https://github.com/BrunoLevy/geogram>, and CGAL at <https://www.cgal.org>.

The gap is not speed. It is an RVD engine you can **read, trust, test, and look inside**, with the knobs reachable, in the style of your own code. mesh-dev’s rule applies: the header says why, the .cpp says how, and nothing hides an idea behind an abstraction.

3 The mathematics the design rests on

Kept to what the design needs. Everything here is standard, and the references at the end have the proofs.

3.1 Cells are intersections of half-spaces

Take seeds $p_1, \dots, p_n$ in $\mathbb{R}^3$, each with a weight w_i (zero for an ordinary Voronoi diagram). The *power distance* from a point x to seed i is

$$\pi_i(x) = \|x - p_i\|^2 - w_i,$$

and the cell of seed i is every point at least as close to i as to any other seed:

$$V_i = \{x : \pi_i(x) \leq \pi_j(x) \text{ for every } j\}.$$

Expand one of those conditions and the $\|x\|^2$ on both sides cancels:

$$\pi_i(x) \leq \pi_j(x) \iff 2x \cdot (p_j - p_i) \leq \|p_j\|^2 - \|p_i\|^2 + w_i - w_j.$$

That is a **half-space**, bounded by the *bisector* plane of i and j . So every cell is an intersection of half-spaces, which makes it a convex polytope: possibly unbounded (a seed on the convex hull of the seeds), and, with weights, possibly empty, or not containing its own seed.

Only a few of the $n - 1$ half-spaces matter: the ones whose plane actually supports a face of the cell. Those seeds are the cell's *neighbours*. For uniformly random seeds in 3D a cell has on average $48\pi^2/35 + 2 \approx 15.54$ faces and $96\pi^2/35 \approx 27.07$ vertices (Meijering, 1953). That number is why the whole problem is cheap. A cell is decided by about fifteen planes out of millions, and finding those fifteen is stage A.

One more fact stage A needs. The bisector of i and j lies at signed distance

$$t_{ij} = \frac{d_{ij}^2 + w_i - w_j}{2d_{ij}}, \quad d_{ij} = \|p_j - p_i\|,$$

from p_i , which is $d_{ij}/2$ without weights (with weights it is negative when p_i itself is on j 's side). A plane farther from p_i than every point of the cell cannot cut it. That is the *security radius* test of §7.1.2.

3.2 Restriction is clipping, piece by piece

The domain Ω is where you want the diagram: a box B , a triangulated surface S , or the inside of a tetrahedral mesh. The restricted cell is $V_i \cap \Omega$. Write the domain as a union of convex *pieces* P_t : the box itself, or the triangles of S , or the tets. Then

$$V_i \cap \Omega = \bigcup_t (V_i \cap P_t).$$

Each $V_i \cap P_t$ is convex, because both factors are. It is exactly *the piece P_t clipped by the half-spaces of cell i* , the same half-spaces as in §3.1. This is the step from Voronoi to RVD, and it is only this. A restricted cell is a union of convex clipped pieces. On a curved or non-convex domain that union can be non-convex, and even **disconnected** (a thin plate: one cell reaches both faces). The traversal in §7.4 is built not to assume otherwise.

Vocabulary: a *restricted Voronoi diagram* (RVD) usually means restricted to a surface. Its volume counterpart is also called the *clipped Voronoi diagram* (Yan et al., 2013). Here both are one thing with two kinds of piece.

3.3 The dual: the restricted Delaunay triangulation

The *restricted Delaunay triangulation* (RDT) records which cells meet on the domain. On a surface, the triangle $\{i, j, k\}$ is in the RDT when $V_i \cap V_j \cap V_k \cap S$ is non-empty. Generically that is a point where the Voronoi edge of i, j and k (part of the line of points equidistant from all three) pierces the surface. In a volume, the tet $\{i, j, k, l\}$ is in it when the Voronoi vertex of those four seeds lies inside Ω . The RDT of a good CVT on a surface is the classic isotropic remesh (Yan et al., 2009). The RDT of a clipped Voronoi diagram is, when the sampling is fine enough, a Delaunay tet mesh of Ω .

When is the RDT a valid mesh of S , with no holes, folds or extra handles? The *closed ball property* (Edelsbrunner and Shah, 1997) answers it. If every restricted cell $V_i \cap S$ is a disk, every pair of cells shares at most one arc, and every triple meets in at most one point, then the RDT is homeomorphic to S . Dense enough sampling guarantees it: an ε -sample does, for small enough ε (Amenta and Bern, 1999). Coarse sampling violates it locally, and that is exactly where remeshing goes wrong. §12.2 turns this property into a tool.

3.4 What is integrated, and why

With a density ρ on Ω (one if you don't care), every restricted cell has

$$\begin{aligned} m_i &= \int_{V_i \cap \Omega} \rho(x) dx && \text{its mass,} \\ c_i &= \frac{1}{m_i} \int_{V_i \cap \Omega} \rho(x) x dx && \text{its centroid,} \\ E_i &= \int_{V_i \cap \Omega} \rho(x) \|x - p_i\|^2 dx && \text{its energy,} \end{aligned}$$

and every pair of neighbours has a *facet measure*: the ρ -weighted area of $V_i \cap V_j \cap \Omega$, or its length on a surface.

The total energy $E = \sum_i E_i$ is the *centroidal Voronoi tessellation* (CVT) energy. Its gradient is $\partial E / \partial p_i = 2 m_i (p_i - c_i)$ (Du et al., 1999), so a CVT, a diagram whose seeds sit at their centroids, is a critical point. **Lloyd's algorithm** is the fixed-point iteration $p_i \leftarrow c_i$. It never raises the energy, but it converges slowly. **L-BFGS** on E with that gradient is far faster, because E is C^2 on convex domains with a smooth density, and in most situations met in practice (Liu et al., 2009).

On a surface, c_i lies off the surface. The *constrained CVT* (Du et al., 2003) keeps each seed on the surface by moving it to the point z of the surface nearest to c_i . That point is exactly the best position on the surface for that cell, because for any z

$$\int_{V_i \cap S} \rho(x) \|x - z\|^2 dx = \int_{V_i \cap S} \rho(x) \|x - c_i\|^2 dx + m_i \|c_i - z\|^2,$$

so Lloyd stays monotone, provided the nearest point is the true, global one (mesh-dev's BVH query returns that). Whether seeds should stay on the surface belongs to the application, and a remesh wants them there. So it is a parameter, not a hidden default.

With a density, CVT cells shrink where ρ is large, with a size proportional to $\rho^{-1/(d+2)}$ in dimension d (Du et al., 1999). That is how an adaptive remesh or sampling is steered.

3.5 Optimal transport: why the weights exist from day one

Give each seed a target mass ν_i , the ν_i summing to the mass of Ω . *Semi-discrete optimal transport* looks for the weights w for which every power cell has exactly its target mass: $m_i(w) = \nu_i$. The mass map $w \mapsto m(w)$ is, up to sign, the gradient of a concave function, the Kantorovich dual, so solving $m(w) = \nu$ is maximising that function. Its Hessian is, again up to sign, a **graph Laplacian on the neighbour graph**, whose edge weights are

$$\frac{\text{the } \rho\text{-weighted area of the facet between } i \text{ and } j}{2 d_{ij}}$$

for the cost $\|x - y\|^2$. So one RVD pass gives the function, the gradient and the Hessian together. A damped Newton method converges globally, under mild conditions on the density (Kitagawa et al., 2019), and Lévy (2015) gives the algorithm in 3D.

Equal-volume cells are what Lagrangian fluid schemes built on power diagrams need: *power particles* (de Goes et al., 2015), and Lévy’s constant-volume free-surface meshes (Lévy, 2022). Blue-noise sampling is optimal transport too (de Goes et al., 2012). That is the reason every data structure below carries a weight per seed even while every weight is zero. Adding them later would mean touching every predicate.

3.6 Degeneracy, stated honestly

Everything above assumes *general position*: no five seeds on one sphere, no domain vertex on a bisector, no surface triangle lying inside a bisector plane. Real inputs break it all the time. Lattices, symmetric seeds, seeds placed on mesh vertices, and a flat face of a CAD surface lying in a bisector plane are all exactly degenerate. A cubic lattice is degenerate at *every* Voronoi vertex, where eight seeds are equidistant.

The design answers this once, in §6.4, with symbolic perturbation. Exact ties are broken as if the input had been moved by an infinitesimal amount, consistently for every predicate. The result is the diagram of that perturbed input. The measures are unchanged, and the combinatorics are generic, sometimes with zero-length edges and zero-area faces where the true diagram has a vertex of high degree. The whole point of §6 is that the algorithms of §7 **may assume general position, and the input never has to provide it.**

4 Architecture

4.1 Layers, and which way they depend

directory	contents
<code>ext/mesh-viewer</code>	the viewer, a submodule, pulled in exactly as mesh-dev does it
<code>src/kernel/</code>	numbers and predicates: filtered doubles, exact fallback, symbolic perturbation, counters. Depends on nothing
<code>src/points/</code>	seeds and weights, the spatial index (nearest seeds), Morton order, samplers (uniform, lattice, on a surface, from a file)
<code>src/domain/</code>	<code>Box</code> , <code>SurfaceDomain</code> , <code>VolumeDomain</code> : convex pieces, their features (vertices, edges, faces), adjacency, an AABB tree
<code>src/cells/</code>	the two clippers, <code>ConvexCell</code> (3D) and <code>ClippedPolygon</code> (on a triangle), and symbolic vertices
<code>src/diagram/</code>	stage A (neighbour oracles) and stage B (traversals)
<code>src/consumers/</code>	display mesh, integrator, restricted Delaunay, validators, statistics
<code>src/optimize/</code>	Lloyd, L-BFGS on the CVT energy, Newton on the weights
<code>src/io/</code>	readers (mesh-dev’s) and writers (<code>.obj</code> , <code>.msh</code> , <code>.csv</code>)
<code>src/app/</code>	the mesh-viewer host: sources, panels, picking, generations
<code>tests/</code>	one source file per suite, with mesh-dev’s <code>check()</code> harness
<code>docs/</code>	this document, and what grows from it

Each directory under `src/` depends only on the ones listed above it, and only `app/` sees the viewer. Nothing outside `app/` includes a GL or ImGui header. The build has the same shape as mesh-dev’s: a static library `rvd_core` (everything but `app/`), the executable `rvd`, and test binaries that link only the core. A machine with no display can build and test all of the mathematics.

4.2 What to reuse from mesh-dev

A lot of what this needs already exists in mesh-dev, written in the style you want here.

mesh-dev piece	what RVD uses it for
mesh/surface_mesh.h, mesh/tet_mesh.h	the domains' pieces and adjacency: fixed connectivity, <code>neighbor(t, e)</code> and <code>neighbor(t, f)</code> , <code>kEdges</code> and <code>kFaces</code> , positively oriented tets, a revision
mesh/vertex_incidence.h io/ (MeshData, the .msh and .obj readers)	the CSR pattern, for the neighbour graph and the grid's bins loading domains, and seeds from a file
mesh/morton_order.h core/hash.h	spatial order for seeds and domain pieces: locality for stages A and B <code>mix64</code> and <code>pow2_at_least</code> , for the open-addressing table that welds symbolic vertices
core/parallel.h geometry/surface_bvh.h, geometry/winding_number.h	<code>deterministic_sum</code> , for every global sum “is this seed inside, and how far from the boundary”: the shortcut that lets interior cells skip restriction entirely (§7.4). The winding numbers follow Barill et al. (2018)
tests/test_support.h ext/lbfgs-lite	<code>check</code> , <code>report_checks</code> , <code>tet_cube</code> , <code>sphere</code> L-BFGS for the CVT

How to take them: at first, point CMake at the mesh-dev sources by path, with an option such as `RVD_MESH_DEV_DIR`, the way `MESH_DEV_VIEWER_DIR` points mesh-dev at another viewer checkout. Compile only the dozen files needed. Do not copy them, because copies drift. If both projects keep needing them, split a small `mesh-core` repository out, the way you split the viewer out on 29 September.

4.3 The stages and what each one produces

stage	reads	produces	cost
seeds	a sampler's parameters, or a file	PointSet : positions, weights, a revision	$O(n)$
A. neighbours	PointSet , and the bounding box B of the domain	NeighbourGraph : for every seed, the seeds whose bisectors bound its cell inside B	$O(nk)$ for k neighbours tried
B. restrict	NeighbourGraph , Domain	a <i>stream</i> of clipped pieces: (seed, piece, symbolic polygon or polyhedron)	$O(\text{output})$
C. consume	that stream	whatever the attached consumers build: display mesh, integrals, facet measures, RDT, validation report, predicate statistics	$O(\text{output})$
D. optimise	integrals and facet measures	new positions (CVT) or new weights (transport)	one A + B + C per iteration

On a box domain, stage B is almost nothing. Each cell from stage A *is* its restricted cell, because the box is the one convex piece. That is the first decision of §1, in table form.

4.4 Stage B streams, and consumers listen

Stage B does not build “the RVD” as a data structure. It hands every clipped piece, as it is made, to the consumers attached to this run, and forgets it. Geogram's RVD works the same way, through callbacks. This matters for three reasons:

- **Size.** The RVD of a million seeds on a million triangles is several million polygons. An optimiser needs only a mass and a centroid per seed, which is 32 bytes.
- **Composition.** A run is “stage B plus a list of consumers”. A Lloyd step attaches the integrator. A display refresh attaches the mesh builder. Validation attaches the validator to either. The traversal is written once.
- **Honesty.** The display mesh is one consumer among others, so the picture cannot show something the integrals did not see.

4.5 Freshness

Every derived structure records the revisions it was built from: the point set’s, the domain’s, and a hash of the parameters that matter. It is refreshed through one `update` that compares them and rebuilds only what is stale. This is `SurfaceQueries::update` in mesh-dev, where asking is the same act as making sure. Here it covers the spatial index, the neighbour graph and the last generation.

4.6 Parallel, and bit-identical

mesh-dev’s rule carries over unchanged: **results are identical on one thread and on sixty-four.** The design keeps it by structure rather than by care:

- Stage A and the default stage B run *per seed*. A seed’s result depends only on the input, and is written into that seed’s own slot. Scheduling cannot change it.
- Every list that feeds a later loop is sorted by seed id before use (neighbour lists, for instance), so traversal order never depends on which thread got there first.
- Every sum over seeds or pieces is a `deterministic_sum`, or is reduced in index order.
- Predicate counters are integers, so the order they are added in does not matter.
- A test runs the same diagram on 1 and on N threads and compares a hash of every output (§10.8).

5 Data structures

Plain arrays indexed by plain integers, as in `TetMesh`, with no attribute store. Everything is double precision, with `glm::dvec3` as three packed doubles (the same `static_assert` as `tet_mesh.h`).

5.1 PointSet: the seeds

field	what	why
positions	one <code>dvec3</code> per seed	the input: never rounded, never recentred
weights	one double per seed, all zero for a Voronoi diagram	power diagrams and transport, without touching a predicate later (§3.5)
revision	bumped on every write, never zero once built	freshness for every cache (§4.5) and for the viewer
permutation	optional Morton order, with old-to-new and new-to-old maps	locality in stages A and B, as <code>MeshPermutation</code> does it. Results go back to the caller’s numbering at the boundary

A seed *is* its index. Duplicate seeds are allowed. Symbolic perturbation gives the later duplicate an empty cell, and the report counts them, so a sampler that produced duplicates is noticed instead of crashing a predicate.

5.2 The spatial index: nearest seeds

A **uniform grid** first. Bin the seeds with a cell size chosen for two to four seeds per bin, stored as CSR (a counting sort, exactly like `VertexIncidence`). A query walks rings of bins outward and returns candidates in increasing distance, and it can be *resumed*: “give me the next 32”. The grid is about a page of code, it is cache friendly, and Morton-ordered seeds are already contiguous inside a bin. It degrades when the density varies by orders of magnitude, which a density-adapted CVT will produce. A kd-tree is the second implementation for that case, behind the same three calls: nearest, k nearest, and resume.

5.3 Domain: where the diagram lives

One interface, three kinds. What every kind provides:

- **pieces**: convex, numbered. The box is one piece, a surface has its triangles, a volume its tets.
- **features** with global ids: vertices, edges (a sorted vertex pair), faces (a sorted triple). These name the vertices of clipped pieces (§5.5), so two pieces that share an edge produce the *same* vertex where a bisector crosses it.
- **adjacency** between pieces, from `SurfaceMesh::neighbor` and `TetMesh::neighbor`, for the element-centric traversal.
- an **AABB tree over pieces**, for “which pieces overlap this box”, which the cell-centric traversal asks (§7.4).
- a **bounding box**, which becomes the box B that stage A clips against. Its coordinates are minima and maxima of input doubles, so they are exact, and no safety margin is needed: touching is decided exactly too.
- optionally a **density** ρ per domain vertex, linear on each piece.

kind	pieces	built from	notes
Box	1: the box	two corners	axis-aligned planes, so their coefficients are exact doubles
SurfaceDomain	triangles	a <code>SurfaceMesh</code>	<code>closed</code> and <code>oriented</code> are passed through to the display
VolumeDomain	tets	a <code>TetMesh</code>	the boundary faces are the surface of the domain

5.4 Constraints: the planes that bound a clipped piece

A *constraint* is a plane with a side, and one of two things:

- a **feature** of the piece: a face of the box, an edge of the triangle (seen as a line in the triangle’s plane), or a face of the tet. What is on the other side is the neighbouring piece, or nothing, on the domain’s boundary.
- a **bisector** between the cell’s own seed i and a seed j . What is on the other side is cell j .

It is stored as a tag and an index, in 32 bits. A constraint *is* the answer to “who is across this edge or face”, which is why the neighbours of a cell, and the facets between cells, fall out of the clipped pieces without being searched for.

5.5 Symbolic vertices: the heart of it

Every vertex of every clipped piece is stored by what defines it, never by its coordinates. On a **triangle** of a surface there are three kinds of vertex, shown in figure 2.

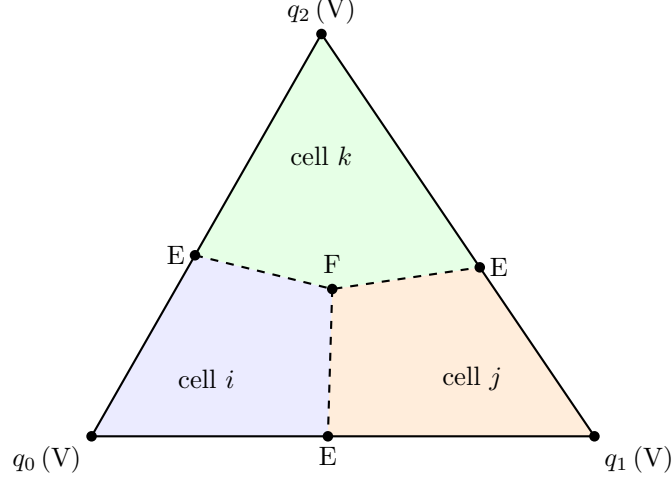


Figure 2: One triangle of a surface, shared by the cells of seeds i , j and k . The corners are **V** vertices. An **E** vertex is where a triangle edge crosses a bisector, where the boundary between two cells (dashed) leaves the triangle. The **F** vertex is where the triangle crosses two bisectors at once: where the Voronoi edge of i , j and k pierces the surface. Every F vertex is one triangle $\{i, j, k\}$ of the RDT.

Cell i 's piece of this triangle is the convex polygon $q_0 \rightarrow E_{\text{bottom}} \rightarrow F \rightarrow E_{\text{left}}$: one V, two E and one F. Cell j 's is $q_1 \rightarrow E_{\text{right}} \rightarrow F \rightarrow E_{\text{bottom}}$, and cell k 's is $q_2 \rightarrow E_{\text{left}} \rightarrow F \rightarrow E_{\text{right}}$. Each E is shared by two cells, and the F by all three. In a **tet**, or in the box, there is a fourth kind, C. The table lists all four, with the number of cells each one bounds and the predicate that tests it against a new bisector (§6.2):

type	defined by	cells	global key	predicate
V	a domain vertex q (a mesh vertex, a box corner)	1	q	side1
E	a domain edge (q_0, q_1) crossing bisector (i, j)	2	(edge, $\{i, j\}$)	side2
F	a domain face or triangle crossing bisectors (i, j) and (i, k)	3	(face, $\{i, j, k\}$)	side3
C	bisectors (i, j) , (i, k) , (i, l) : a Voronoi vertex	4	$\{i, j, k, l\}$	side4

The **global key** is the vertex's defining data, normalised: the domain feature it lies on (a vertex, an edge or a face, and none for C), together with the sorted set of seeds whose cells it bounds (just the owning cell for V, so the feature alone suffices). It comes out the same from every cell and every piece that sees the vertex. Four things follow from that:

1. **Welding is exact.** Vertices are merged by key in a hash table (`mix64`, open addressing, as in `TetMesh`'s face table). There is no tolerance and no “merge within 10^{-9} ”, and so no cracks and no false merges.
2. **The dual is free.** Every F key on a surface *is* an RDT triangle $\{i, j, k\}$. Every C key in a volume is a Delaunay tet $\{i, j, k, l\}$.
3. **Validation is combinatorial.** A key of type V, E, F or C must turn up in exactly 1, 2, 3 or 4 cells, namely those of its seed set. Anything else is a bug you can point at (§7.5).
4. **Debugging can answer “why”.** Click a vertex and the tool can say which seeds and which feature made it, and how the predicate that kept it was decided.

Each vertex also carries a **constructed position** in double, computed from its definition by a small linear solve. It exists for display and integration only. **No predicate ever reads it** (§6.1).

5.6 ClippedPolygon: a triangle clipped by one cell

A convex polygon in the triangle’s plane, stored as a cycle of (vertex, constraint) pairs, where constraint k supports the edge from vertex k to vertex $k + 1$. It starts as the triangle: three V vertices and three edge constraints. Clipping by a bisector classifies the vertices. The vertices on the wrong side form one contiguous run, because the polygon is convex. The run is replaced by two new vertices: (previous constraint $\cap$ bisector) and (bisector $\cap$ next constraint). A clipped triangle rarely has more than eight vertices, so it lives in a small inline buffer that allocates only in the rare case it must grow.

The edges tell you everything else. An edge on bisector (i, j) is a piece of the boundary between cells i and j on the surface: its length is a facet measure, and j is a neighbour. An edge on a triangle edge continues into the adjacent triangle, or it is the surface’s border.

5.7 ConvexCell: a box or tet clipped by one cell, stored as its dual

This is the structure of Ray et al. (2018), and it is what makes a 3D clipper short enough to trust. The polytope is stored as:

- **planes:** an array of constraints. Six box faces or four tet faces to start with, then the bisectors as they cut.
- **vertices:** an array of *triangles* (a, b, c) , each three indices into the planes. A vertex of the polytope is where exactly three planes meet, so it *is* a triangle of plane indices: the polytope’s dual. Each vertex also carries its symbolic key and its constructed position.

To **clip by a plane** P : mark the vertices on the wrong side of P , using exact predicates. If none is marked, P does not cut. If all are, the cell is empty. Otherwise the marked triangles form one patch in the dual, whose boundary is a single cycle of dual edges (a, b) . Add a new triangle (a, b, P) for each boundary edge, and delete the marked ones. That is the whole operation. Planes that no triangle mentions any more are faces that were cut away.

A concrete case. A cube is 6 planes and 8 triangles: an octahedron in the dual. Cut one corner off. Its triangle goes, and the three dual edges around the hole each get a new triangle with P . Now it is 7 planes and 10 vertices, and $V - E + F = 10 - 15 + 7 = 2$, as it should be.

Why this and not a half-edge polyhedron:

- It is **small**: a Voronoi cell averages 27 vertices, so it fits on the stack.
- It has **no pointer bookkeeping** to get wrong.
- It is **simple by construction**: symbolic perturbation guarantees that exactly three planes meet at each vertex, which is the only thing the representation assumes.
- **Every vertex’s symbolic definition is its triangle**, for free.

A face, for display or integration, is the cycle of triangles around one plane index.

5.8 NeighbourGraph

CSR: $n + 1$ offsets and the neighbour ids, each list sorted by id (that is the determinism rule of §4.6). It is symmetric by construction when the predicates are exact, and validated as such (§7.5). Parallel arrays can hang off its entries: the facet measure and facet centroid, filled by the integrator, which is exactly what the transport Hessian (§3.5) and a Laplacian on the seeds need.

5.9 Generation: one result, frozen

The unit the app publishes. It is immutable and held by `shared_ptr`, and contains everything needed to show one state: the display meshes, the carrier meshes for overlay channels, the channel

index arrays, per-cell statistics, the validation report, the predicate counters, and the revisions it was computed from. A computation builds a new one. The host swaps the pointer between frames, and the viewer keeps the old one alive through `MeshView::keep_alive` until its layers are gone. §8.4 has the details.

6 Predicates and robustness

This section is why the tool can be trusted, and it is also the most interesting part of the design to look at while it runs.

6.1 The one rule: predicates read input numbers only

Every decision the clippers take is a *predicate*: a question with a yes-or-no answer, such as “is this vertex on seed i ’s side of the bisector of i and m ?”. The rule is that a predicate is evaluated from the **input numbers only**: seed coordinates, weights, and domain vertex coordinates, exactly as given. It never uses a constructed coordinate, such as the rounded position of a vertex computed earlier.

Why so strict? A constructed vertex carries rounding error, and two computations that look at the same vertex through different roundings can disagree. Say cell i decides that the bisector with j cuts off a sliver of it, while cell j decides that the same facet has zero area. Now i lists j as a neighbour and j does not list i . The tiling has a crack or an overlap. The dual has a missing triangle. In a propagation, it can loop forever. Floating-point Voronoi codes live with this through tolerances. This one removes it. Every predicate is the sign of a polynomial in the input, and the sign of a polynomial in doubles can be computed exactly.

6.2 The predicate family

Write

$$f_{ia}(x) = \pi_i(x) - \pi_a(x) = 2x \cdot (p_a - p_i) - c_a, \quad c_a = \|p_a\|^2 - \|p_i\|^2 + w_i - w_a,$$

for the difference of power distances. It is an *affine* function of x (the $\|x\|^2$ cancels, §3.1): negative on i ’s side of the bisector of i and a , positive on a ’s. Each clipping question is “what is the sign of f_{im} at this vertex?”, for the cell of seed i and a new bisector (i, m) . The vertex types of §5.5 give four predicates:

predicate	the vertex it tests	the sign it computes
side1	a domain vertex q	$f_{im}(q)$. Degree 2
side2	where the domain edge (q_0, q_1) crosses bisector (i, j)	a ratio of 2×2 determinants of f values at q_0 and q_1 . Degree 4 over degree 2
side3	where the domain triangle (q_0, q_1, q_2) crosses bisectors (i, j) and (i, k)	a ratio of 3×3 determinants of f values at the three corners. Degree 6 over degree 4
side4	the Voronoi vertex of i, j, k, l	the power in-sphere test on the five seeds over orient3d of four. Degree 5 over degree 3
orient3d, in_power_sphere	(not a clipping question)	the Delaunay predicates, for the optional Delaunay (§7.1.3) and for validation

The first three share one shape, and it is worth seeing why. A vertex where k bisectors cross a k -dimensional domain feature (a vertex, an edge or a triangle: $k = 0, 1, 2$) has barycentric coordinates in that feature. Because every f_{ia} is affine, Cramer’s rule turns “the value of f_{im} at that vertex” into a ratio of two $(k + 1) \times (k + 1)$ determinants of f values at the feature’s corners:

the denominator has a row of ones, and the numerator has the row of f_{im} values in its place. Written out,

$$\begin{aligned}
\text{side1: } f_{im}(x) &= f_{im}(q), \\
\text{side2: } f_{im}(x) &= \frac{\begin{vmatrix} f_{im}(q_0) & f_{im}(q_1) \\ f_{ij}(q_0) & f_{ij}(q_1) \end{vmatrix}}{\begin{vmatrix} 1 & 1 \\ f_{ij}(q_0) & f_{ij}(q_1) \end{vmatrix}}, \\
\text{side3: } f_{im}(x) &= \frac{\begin{vmatrix} f_{im}(q_0) & f_{im}(q_1) & f_{im}(q_2) \\ f_{ij}(q_0) & f_{ij}(q_1) & f_{ij}(q_2) \\ f_{ik}(q_0) & f_{ik}(q_1) & f_{ik}(q_2) \end{vmatrix}}{\begin{vmatrix} 1 & 1 & 1 \\ f_{ij}(q_0) & f_{ij}(q_1) & f_{ij}(q_2) \\ f_{ik}(q_0) & f_{ik}(q_1) & f_{ik}(q_2) \end{vmatrix}},
\end{aligned}$$

and the vertex is on i 's side when the ratio is negative. So **side1**, **side2** and **side3** are one formula at three sizes: one thing to derive, one thing to filter, and one thing to test. **side4** has no domain feature, and comes from solving the three bisector equations directly. With $u_a = 2(p_a - p_i)$ and c_a as above,

$$\text{side4: } f_{im}(x) = - \frac{\begin{vmatrix} u_j & c_j \\ u_k & c_k \\ u_l & c_l \\ u_m & c_m \end{vmatrix}}{\begin{vmatrix} u_j \\ u_k \\ u_l \end{vmatrix}},$$

where each u_a is a row of three numbers, so the numerator is 4×4 and the denominator 3×3 . The derivations belong in the header of `kernel1/`, written out in full, in the style of mesh-dev's `winding_number.h`.

A denominator is never zero for a vertex that the clipper actually created, provided the domain pieces are not degenerate (§6.4). Its sign is part of the answer.

6.3 Fast when easy, exact when not

Each predicate is evaluated in up to three steps:

1. **Filter.** Evaluate the determinants in double, together with a certified bound on their rounding error: a static or semi-static filter, as in [Shewchuk \(1997\)](#) and [Meyer and Pion \(2008\)](#). If a value is farther from zero than its bound, its sign is certain. On random input this settles almost every call.
2. **Exact.** Otherwise evaluate the same polynomials exactly. Start with GMP rationals (`gmpxx`, already installed here). They are the simplest thing that is obviously correct. Shewchuk-style expansion arithmetic is the faster replacement, if profiling on degenerate inputs ever says the exact path matters. On random inputs it will not. On a lattice it will, because every vertex goes this way.
3. **Symbolic perturbation**, if the exact value is zero: next section.

6.4 Ties: symbolic perturbation

Imagine every seed's weight increased by an infinitesimal amount ε_i , with $\varepsilon_0 \gg \varepsilon_1 \gg \varepsilon_2 \gg \dots$. Seed 0's extra weight is infinitely larger than seed 1's, which is infinitely larger than seed 2's, and so on, but every one of them is infinitesimal. This is Simulation of Simplicity (Edelsbrunner and Mücke, 1990), in the spirit of how Geogram's PCK (Lévy, 2016) handles the same predicates. Under that imaginary input, no predicate is ever exactly zero. When the true value is zero, its sign is the sign of the first non-zero term of its expansion in the ε s, taken in order of priority.

Because perturbing a weight shifts each f_{ia} by a constant $(\varepsilon_a - \varepsilon_i)$, each such term is one of the determinants above with a row replaced by ones: a smaller predicate of the same family. The chain always ends in a term that cannot be zero. For `side1` that is a comparison of two indices. For the others, the coefficient of the tested seed's ε_m is, up to sign, the vertex's own denominator, which is non-zero because the vertex exists.

What that buys:

- **Consistency.** Every predicate answers questions about *the same* imaginary input, so all decisions agree with each other, and the output is a valid complex: the diagram of the perturbed seeds.
- **No special cases in §7.** A clipper never meets a vertex exactly on a plane, and a `ConvexCell` vertex always has exactly three planes.
- **Honest geometry.** Measures are unaffected, because the perturbation is infinitesimal. A cube corner of a cubic lattice, where eight cells meet at one point, becomes several ordinary vertices at the same position, joined by zero-length edges, which integrate to nothing.

Three things to keep in mind:

- The domain is not perturbed. Degenerate pieces (zero-area triangles, flat tets) are removed when the domain is built. They are counted and can be shown, never silently dropped.
- The tie-break follows the seed index. With Morton order on or off, degenerate inputs get different, equally valid combinatorics. The measures are the same either way.
- A future Delaunay must use the same perturbation, or it will disagree with the clippers exactly on degenerate input, which is where agreement is the point.

6.5 The oracle: a second, stupid implementation

For every predicate there is also a reference that does the obvious thing. It constructs the vertex exactly, in GMP rationals, by solving its defining linear system. Then it compares the two power distances exactly. There are no determinants to derive, no filters, and no perturbation: it returns zero on an exact tie. It is slow, and it is correct by inspection.

The tests (§10.2) compare every fast predicate to the oracle wherever the oracle is non-zero, on millions of random and near-degenerate inputs. Where the oracle is zero, they check the perturbed answer for consistency instead: antisymmetry, and agreement between predicates about the same vertex. `mesh-dev` compares its BVH to Geogram's for the same reason: two implementations built from different starting points should never disagree.

6.6 Counters and switches

Every predicate counts its calls and how each was decided: by the filter, exactly, or by perturbation. The counters are thread-local and summed at the end. They are integers, so the sum is deterministic. The app shows them (§8.6), and two switches go with them:

switch	effect	what it is for
exact everywhere	skip the filter	the result must be <i>bit-identical</i> . If it is not, a filter is wrong
floating point only (unsafe)	filter results trusted even when uncertain, zeros left unresolved	to see what goes wrong without §6. On a cubic lattice, expect asymmetric neighbours and a broken restricted Delaunay triangulation, found by the validators and shown in the viewer

6.7 Constructions: numbers for looking at, not for deciding

Positions are still needed, to draw the cells and to integrate over them. Each vertex’s position is computed in double from its definition: the barycentric coordinates of the same Cramer ratios for E and F vertices, and a 3×3 solve for a Voronoi vertex. Integrals are then accurate to rounding, around 10^{-15} relative, which is far below anything a CVT or a transport solver can see. And a mistake here can only make a picture or an integral slightly wrong. It can never make the structure inconsistent, because no decision reads these numbers.

7 Algorithms

7.1 Stage A: which half-spaces bound each cell

Three neighbour oracles, built in this order. The first is the test reference, the second the default, and the third an independent second opinion.

7.1.1 Brute force (the reference)

For each seed, start from the box B and clip it by the bisector with *every* other seed. That is $O(n^2)$, which is useless beyond a few thousand seeds, but it has no stopping rule, no search structure and no cleverness to be wrong. It is what §10 compares the others against.

7.1.2 Nearest seeds and a security radius (the default)

This is the algorithm of Lévy and Bonneel (2012) and Ray et al. (2018), made exact. For each seed i :

1. $C \leftarrow$ the box B , as a **ConvexCell**.
2. For each seed $j \neq i$, nearest first (from the grid, 32 at a time, resumed as needed):
 - (a) $R \leftarrow$ the largest distance from p_i to a vertex of C , rounded up.
 - (b) If the bisector of i and j lies farther than R from p_i : **stop**. No farther seed can cut C either, so the cell is certified.
 - (c) Clip C by the bisector of i and j .
3. $N(i) \leftarrow$ the seeds whose bisectors are still faces of C .

The test in step 2(b) is the one of §3.1: without weights, stop once $d_{ij} > 2R$. With weights, replace w_j in t_{ij} by the largest weight in the set. The resulting bound $(d^2 + w_i - w_{\max})/(2d)$ grows with the distance d , so once it exceeds R for the next seed it does so for every farther one, and it can only make the test stop later than the true weights would, never earlier. R is computed from constructed positions, so it is enlarged by a margin. **The certificate may be pessimistic, never optimistic.** Clipping by one bisector too many costs a little time. Stopping one too early would silently lose a face. Distance ties are ordered by index, so the order of clipping never depends on anything but the input.

Why this is the default:

- It is **the same code as the RVD**: a `ConvexCell` clipped by bisectors. The Voronoi maker and the RVD engine are one program from the first day.
- It is **embarrassingly parallel**: one seed per task, no shared state.
- It is **local**. Any single cell can be computed on its own, typically in microseconds, without building anything global. That makes inspecting one cell of a ten-million-seed diagram instantaneous, and it is what cell replay (§12.3) runs on.
- Weights, periodic boxes and anisotropy (§11.3) change only the distance order and the certificate, never the structure.

Its weak point is strongly non-uniform density, where the grid returns neighbours slowly. That is the kd-tree’s job (§5.2). With exact predicates the graph comes out symmetric, $j \in N(i)$ exactly when $i \in N(j)$, even though every cell was computed alone. Floating-point versions of this algorithm cannot promise that.

7.1.3 Delaunay (optional, later)

Incremental Bowyer–Watson, with an infinite vertex for the convex hull, walking point location, and insertion in biased randomised order over Morton-sorted seeds (Amenta et al., 2003). It needs `orient3d` and `in_power_sphere` with the **same** perturbation as §6.4; Devillers and Teillaud (2011) work out perturbations for exactly this case, weighted Delaunay in 3D. It gives the full, unbounded neighbour graph and every Delaunay tet.

It comes late because it is the largest single piece of code in the project, and nothing in stages B to D needs it. It comes at all because an independent algorithm for the same graph is the strongest test there is: restricted to B , the Delaunay graph and the meshless graph must be *identical*, on every input, degenerate ones included. A weighted Delaunay triangulation is also an output worth having.

7.2 The 3D clipper

The `ConvexCell` of §5.7 starts from the box, or from a tet. Clipping by a bisector runs the predicate that matches each vertex’s type, and the type is read off the vertex’s three planes:

the three planes of a vertex	type	predicate
three features (a box corner, a tet vertex)	V	<code>side1</code>
two features and a bisector (on a domain edge)	E	<code>side2</code>
one feature and two bisectors (on a domain face)	F	<code>side3</code>
three bisectors (a Voronoi vertex)	C	<code>side4</code>

Per cell, the clipper also provides the faces (the cycle of vertices around each plane), R for the certificate, the volume, the first and second moments (by splitting into tets from one vertex), and the area and centroid of every bisector face. A clip costs one predicate per vertex, and a cell has about 27 vertices, so it is a few dozen predicate evaluations per bisector.

7.3 The polygon clipper

The `ClippedPolygon` of §5.6 does the same in a triangle’s plane. A vertex’s type is read off its two constraints: two edges make a V, an edge and a bisector an E, two bisectors an F. It provides the area and moments (from triangles fanned around one vertex), and the length and midpoint of every bisector edge.

7.4 Stage B: which pieces each cell meets

This is where Voronoi becomes RVD, and there are two ways to organise it.

	cell-centric (the default)	element-centric (Geogram’s way)
outer loop	over seeds	over domain pieces
how it finds work	ask the AABB tree for the pieces that overlap the bounding box of $V_i \cap B$, and clip each one by the bisectors of $N(i)$	start from the seed nearest a corner of the piece (nearest in power distance, found by walking the neighbour graph from the Euclidean nearest), which owns that corner. Clip, and every bisector edge of the result names another cell that also meets the piece. Visit it, until none is new
complete?	always. A piece that meets the cell overlaps its box, however disconnected the restricted cell is	always. Pieces are convex, so the cells tile each one in a connected pattern
output grouped by	seed, so integrals need no reduction	piece, so integrals are reduced by seed afterwards
parallel over	the n seeds	the pieces
best when	always good. Essential when seeds outnumber pieces (a cube with 12 triangles and 10^5 seeds)	pieces outnumber seeds by far, and the pieces that overlap a box but miss the cell cost too much

Build the cell-centric one first. It is complete by an argument you can check in one line, and it keeps each seed’s work in one task. Rejecting a piece that overlaps the box but misses the cell is cheap: most fail on the first plane, if the planes are tried nearest seed first. The element-centric traversal is the optimisation to add later if the profile asks for it, and the two must produce identical results (§10.5). A third organisation, which validates each corner of the RVD on its own (Sainlot et al., 2017), is worth reading before M8.

The shortcut that makes volumes cheap. In a volume, most cells never touch the boundary. If seed i is inside the domain (winding number above one half) and the ball of radius R around p_i does not reach the boundary surface (`nearest_within` with R^2 , in mesh-dev’s `SurfaceQueries`), then $V_i \cap \Omega$ is $V_i \cap B$, and no tet is needed at all. Only the boundary layer of cells is clipped against tets. This is the same idea as the clipped Voronoi diagram of Yan et al. (2013), and it makes a volume cost about as much as a box. The same test against a surface empties the restricted cell of any seed whose ball misses the surface entirely.

7.5 Stage C: the consumers

Each consumer receives (seed, piece, clipped polygon or polyhedron) and keeps what it needs.

Mesh builder, for display and export.

- On a surface, every clipped polygon is a face. In a volume, a face is kept if it lies on a bisector or on the domain boundary. A face on a tet face shared with another tet of the *same* cell is internal to the cell and is dropped. Its constraint says so: it is a tet face whose neighbouring tet is also in cell i .
- Faces are fanned into triangles. Vertices are welded by symbolic key (§5.5), and every triangle records its cell.
- The adjacency of the restricted cells (cells sharing an arc on a surface, or a face in a volume) is coloured greedily, in smallest-last order. When the cells tile a sphere-like surface as disks, that graph is planar, and smallest-last greedy never needs more than six colours on a planar

graph. In a volume it takes more, typically around ten. The triangles are split by colour class into one mesh per class, the *chromatic layers* that §8.2 publishes, and the host gives each class its own colour (the viewer’s default palette repeats after eight). No two cells that share an arc or a face share a colour.

- It also builds the **skeleton**: all welded vertices, the Voronoi edges (polygon edges on bisectors, polyhedron edges between two bisector faces), and each vertex’s type and how its predicate was decided. That is what the overlay channels index.

Integrator. Per seed, the mass, first moment and second moment about p_i , with closed forms on triangles and tets, including a linear density. Per neighbour pair, the facet measure and centroid. Everything goes into per-seed slots, and totals are **deterministic_sums**.

Restricted Delaunay builder. On a surface, every F key is a triangle $\{i, j, k\}$, oriented by the domain triangle it was found in. In a volume, every C key is a tet $\{i, j, k, l\}$, and the F keys on the domain’s boundary faces are its boundary triangles. A triple that appears under two different F keys means a Voronoi edge pierces the surface twice: a closed-ball violation, and a place where the RDT will fold. It is recorded, not hidden.

Validators, run from the tests and from a button in the app.

check	what must hold	what a failure means
coverage	$\sum_i m_i$ equals the domain’s measure, to about 10^{-12} relative	a hole or overlap in the tiling
key census	every V, E, F, C key turns up in exactly 1, 2, 3, 4 cells: those of its seed set	an inconsistent predicate, or a clipper bug, located exactly
symmetry	$j \in N(i) \iff i \in N(j)$	a cell computed wrongly on one side
facet agreement	the facet (i, j) has the same measure seen from i and from j	the same, measured
nearest-seed sampling	at random points of the domain, the brute-force nearest seed (in power distance) is the cell that contains the point	the diagram is not the Voronoi diagram
cell topology	each restricted cell is one disk: one component, one boundary loop, $\chi = 1$	the closed ball property fails there (§12.2). Not a bug, a sampling fact
RDT manifoldness	on a closed surface, every RDT edge is in exactly two RDT triangles, and $\chi(\text{RDT}) = \chi(S)$	the remesh is not a valid mesh of S

Every failure is reported with its indices, so the app can show it as an overlay channel.

Statistics. Per cell: counts of faces, edges and vertices by type, the mass, and the dimensionless second moment

$$G = \frac{1}{d} \frac{\int_{\text{cell}} \|x - c\|^2 dx}{\text{vol}^{1+2/d}},$$

with c the cell’s centroid and the density taken as one. It measures a cell’s quality independently of its size: $1/12$ for a cube, and 0.0785 for the truncated octahedron of the BCC lattice, the best known in 3D. Histograms go to the panel.

7.6 Stage D: the optimisers

Each iteration is one pass of A, B and the integrator, and each iteration can become a generation for the viewer (§8.4), so every optimiser can be watched.

- **Lloyd.** $p_i \leftarrow c_i$. On a surface, optionally move c_i to the nearest point of the surface, from mesh-dev’s BVH (§3.4). The energy is recorded at every step. Stop when its relative decrease, or the largest step measured in cell sizes, falls below a tolerance.
- **L-BFGS** on the CVT energy, with gradient $2 m_i (p_i - c_i)$, using `lbfgs-lite` from mesh-dev. Every line-search trial is one more pass. On a surface, the gradient is projected onto the tangent planes and the positions back onto the surface.
- **Newton on the weights** for transport. Masses and facet measures give the gradient and the graph-Laplacian Hessian of §3.5. Solve with Eigen: conjugate gradients with a Jacobi preconditioner, or a sparse LDL^T at moderate sizes. Fix one weight, because shifting every weight by the same amount changes nothing. Damp the step so that no cell’s mass falls below a floor, the safeguard of Kitagawa et al. (2019). Stop when the largest $|m_i - \nu_i|$ is below a tolerance.

8 On top of mesh-viewer

The viewer needs no change for anything before milestone M6 (§9). This section maps the design onto its actual interfaces: `MeshSource`, `OverlaySource`, `Panel`, `defer`, revisions and `keep_alive`. Two small, general extensions for later are in §8.8.

8.1 The host, in the viewer’s own terms

It follows the “Writing a host” part of the viewer’s README, and `examples/demo.cpp` is the template.

- **RvdSession** (core side, no GL) owns the domain, the seeds, the parameters and the current generation. It is the counterpart of mesh-dev’s `Document`.
- **RvdSource**, a `MeshSource`, and **RvdOverlay**, an `OverlaySource`, are thin adapters over the current generation, the counterpart of mesh-dev’s `document_source.h`. Nothing is copied: a `MeshView` points into the generation’s arrays.
- **Panels** hold the controls (§8.6).
- **main** declares the session before the viewer, so it outlives the GPU copies. A dropped `.obj` or `.msh` becomes the domain, and a dropped point file becomes the seeds, through `set_drop_handler`.

8.2 What is published

mesh	what it holds	why it is shaped like this
domain	the domain’s vertices, and its surface triangles, or a volume’s hull and its tets, or the box’s 12 triangles. Stable id; the revision changes when the domain moves	listed first: the first mesh fixes the scene origin, and <code>mesh_center</code> reads triangles only, so a points-only mesh first would put the origin at zero. On a surface the host hides it while cells are shown, because the cells lie exactly on it and two coplanar layers z-fight
seeds	the seed positions, and no triangles. Stable id; the revision changes whenever the seeds move, every Lloyd step	a <i>carrier</i> : it draws nothing, and holds the vertices the seed channels index. <code>closed = false</code>

mesh	what it holds	why it is shaped like this
cells, class k (for $k < K$)	the vertices of that class's cells, copied per cell, and the faces of those cells. A new id per generation; the revision changes when the shrink slider moves	one colour per layer, so one layer per colour class (§7.5). Copies per cell make “shrink cells toward their centroid” a position-only change: interactive, with no new ids. In 3D, <code>closed = true</code> and <code>oriented = true</code> (faces wound outward): each layer is a union of closed polyhedra, so the viewer's stencil cap fills every cut cell's cross-section in a shade of its class colour, for free
skeleton	the welded vertices of the whole diagram, and no triangles. A new id per generation	the carrier for the Voronoi edges and every vertex channel. A channel indexes one mesh, and the edges between two classes belong to neither class layer
RDT	the generation's snapshot of the seed positions, and the RDT triangles; in a volume, the RDT tets (positively oriented) and their hull. A new id per generation	the remesh, in <i>shaded + edges</i> mode. In a volume, the viewer's cutting plane shows the Delaunay tets standing in the cut, and <code>tet_labels</code> can mark slivers, so by label colours them
inspector	an icosphere, a quad, and the replayed cell. The first two have stable ids and new positions when the selection changes; the replayed cell gets a new id per step	the security sphere, the bisector being clipped, and the partial cell of §12.3

8.3 Overlay channels

A fixed list, always present, emptied rather than removed. The viewer keeps a channel's colour and checkbox while the same name stays in the same slot, so a stable order is what makes the user's settings survive every run. This is mesh-dev's `DebugDraw::reset` rule.

channel	primitive	indexes	shows
seeds	points	seeds	every seed
selection	points	seeds	the selected seed and its neighbours
neighbour graph	lines	seeds	the pairs (i, j) , $i < j$: the Delaunay edges whose Voronoi facet meets B
Voronoi edges	lines	skeleton	the diagram's edges, which the fanned triangles' own edges would clutter
decided exactly	points	skeleton	vertices where a filter was not enough: the robustness microscope
decided by perturbation	points	skeleton	vertices that exist only thanks to §6.4, where the input is degenerate
closed-ball violations	points	skeleton	cells that are not disks, pairs sharing two arcs, Voronoi edges piercing twice
validator failures	points	skeleton	should be empty, always. Any point here is a bug with an address

Channels draw even when their layer is hidden, and **in front** lifts them over opaque cells, which is exactly what the seed and skeleton carriers need.

8.4 Generations, ids, and keeping what the user set

The viewer uploads a mesh's triangles once, the first time it sees its id, and a host must never reuse an id. A new diagram has new connectivity, so it gets new ids from a counter in the host. Each generation is an immutable object, and every `MeshView` from it carries `keep_alive`, the

generation's `shared_ptr`. The viewer then holds the generation for as long as any layer shows it, and there is no destruction order to think about.

A new id is a new **Layer**, with default colour, alpha and visibility. Left alone, every Lloyd step would undo what the user set in the View window. So the host **carries the style over** when it installs a generation:

1. read each outgoing layer's settings through `viewer.scene().layer(old_id)`: colour, alpha, visibility, cap and solid cap;
2. swap in the new generation;
3. call `viewer.sync()`, which is public, so the new layers exist now rather than at the end of the frame;
4. write the settings into `viewer.scene().layer(new_id)`, with class *k* inheriting from class *k*.

The first time a class layer appears there is nothing to inherit. The host then gives it its class colour and turns its **cap** on, which the viewer leaves off by default. That is what makes a cut through 3D cells read as a section of solid cells rather than of hollow shells.

One thing this cannot keep is a custom **composite order**: `Scene::sync` rebuilds the order from scratch when a layer disappears. So in phase one the source lists its meshes in the order that composites best, and the topology revision of §8.8 is the real fix.

The one mutable thing is **display positions**. The class layers' positions are a host-owned array derived from the generation, with each cell scaled toward its centroid by the shrink factor. Moving the slider rewrites that array and bumps the revision: same ids, no new generation, and the viewer re-uploads positions only.

8.5 The frame, and where the work happens

The viewer's frame order decides where each kind of work goes.

the viewer's frame, in order	what RVD does there
wait for input, or poll while a panel is <code>animating()</code>	
ImGui frame: each open window's <code>Panel::ui</code>	widgets and parameters only. A button calls <code>viewer.defer(compute)</code> , and never computes
run what the panels deferred	a one-off run: A, B, C, and a new generation
<code>Panel::update</code> on every panel	a running optimiser: one iteration per frame (or a time budget), then the new generation and the style carry-over of §8.4
<code>Scene::sync</code> , then the overlay's <code>Renderer::draw</code>	new layers appear, old ones go

- **One-off runs** are deferred, so they never run inside an open ImGui window and never change the meshes halfway through drawing the windows that describe them. That is the viewer's own rule, from `viewer.h`.
- **Optimisers** step in `update` and return `animating() == true` while running, so the viewer polls instead of sleeping. A Lloyd loop is a clock, like the demo's wobble.
- **Big diagrams** go to a worker thread. It snapshots the seeds, computes a generation, and `update` swaps it in when it is ready, while the viewer keeps drawing the last one. Generations are immutable, so nothing is shared mutably across threads. A cancel flag is checked between seeds.

8.6 Windows

Four panels:

window	contents
Diagram	the domain (box, surface, volume; from a file or a generator). The seeds (count, sampler, random seed, weights). The neighbour oracle (brute force, meshless, Delaunay) and the traversal (cell- or element-centric). The display (shrink, classes, skeleton). run
Optimise	Lloyd, L-BFGS or transport. Iterations per frame. Live plots of the energy and the gradient norm (<code>ImGui::PlotLines</code>). Start and stop
Cell	the selected cell: seed, weight, mass, centroid, G , counts by vertex type, and its neighbours as buttons that select them. The replay controls (§12.3). The security sphere
Robustness	predicate counters per predicate and per decision path. The two switches of §6.6. validate , with each failure listed next to the channel that shows it. The closed-ball summary

Keys: the viewer owns Esc, F, W and P. The host takes **C** (select the cell under the cursor), **[** and **]** (replay one clip back or forward), and **L** (start or stop the optimiser).

8.7 Picking a cell

The viewer has no click hook for hosts, but everything needed is public. On **C**, read the mouse position from `ImGui`’s IO. It is in window units, so multiply by `DisplayFramebufferScale` to get the framebuffer pixels the camera works in, then call `Camera::screen_ray`. The ray is in *scene* space, so add `Scene::origin()` to get file space. Intersect it with the class layers’ triangles, skipping hidden layers and hits on the clipped side of the cutting plane. Brute force is fine for one click, even over millions of triangles. The hit triangle’s cell id is the selection.

8.8 Where the viewer pinches, and two small extensions

Phase one lives with four things:

1. **One colour per layer**, hence the chromatic layers.
2. **Connectivity fixed per id**, hence new ids per generation, the style carry-over, and a composite order that resets.
3. **No click hook for hosts**, hence picking on a key.
4. **Label colours only for tets in the cut**, not for triangles.

Two extensions would remove the first two. Both are general, not specific to RVD, and both keep the viewer’s “positions only” vertex layout:

- **(a) Per-triangle labels.** `MeshView` gains one small integer per drawn triangle, under the existing `labels_revision`. The surface shader looks the label up by `gl_PrimitiveID` in a texture buffer, both core in GL 3.3, and takes its colour from the layer’s `LabelStyle` list, editable in Inspect as tet labels already are. Then one layer carries the whole diagram, coloured per cell class or per anything: materials, patches, a segmentation in mesh-dev.
- **(b) A topology revision.** `MeshView` gains a `topology_revision`. When it changes, `Layer::refresh` re-uploads the index buffer and re-derives the drawn vertices, but keeps the layer, its settings and its place in the composite order. That removes the id churn and the carry-over code, and helps any host that remeshes.

Neither is required before M6, and both are small enough to review in one sitting. A click hook (`Panel::on_click` with a scene-space ray) is the third candidate, with the lowest priority.

9 Implementation plan

Ten milestones, in order. Every one ends with a test suite that proves it and a picture that shows it. Nothing is built before the thing that checks it.

M0. The skeleton

- CMake with `rvd_core`, `rvd` and the test binaries. `ext/mesh-viewer` as a submodule. The `RVD_MESH_DEV_DIR` option (§4.2). `-DRVD_BUILD_VIEWER=OFF` for headless machines.
- The host: an `RvdSession`, the two sources, the four empty windows. A box domain and uniform random seeds, regenerated by a deferred button.
- **Done when** the app shows a box with a point cloud in it, and `ctest` runs an empty suite with no display.

M1. Predicates

- `kernel/`: `side1` to `side4`, `orient3d`, `in_power_sphere`, each filtered, exact with GMP, and perturbed (§6). The oracle. The counters.
- **Tests**: `predicates` (§10.2).
- **Done when** millions of random and near-degenerate cases agree with the oracle, and the perturbed answers pass the consistency checks. You will see nothing new yet, and that is right: this is the foundation, and it is proven before anything stands on it.

M2. One cell, then all of them the slow way

- `ConvexCell` (§5.7): from a box, clip, faces, moments. The brute-force oracle (§7.1.1) for every seed. The mesh builder with chromatic layers, the skeleton, the Voronoi-edges channel, the `Cell` window.
- **Tests**: `convex_cell` (a cube's corner cut, a tet, hand-checked cuts), and `voronoi_box` (§10.3, with the lattices of §10.4).
- **You see** the first Voronoi diagram: coloured cells, cut open by the plane with their cross-sections capped. Then the cubic lattice, where every vertex needs §6.4, and the Kelvin cells of the BCC lattice.

M3. Fast Voronoi in a box

- The grid (§5.2), the security radius (§7.1.2), OpenMP over seeds, and the deterministic collection of results.
- **Tests**: meshless against brute force (§10.5), the Meijering statistics (§10.4), thread independence (§10.8), and the first benchmark (§10.9).
- **You see** a diagram of 10^5 seeds drawn whole, a 10^6 -seed one computed and inspected cell by cell (drawn whole it would be tens of millions of triangles, too many for a laptop GPU), the Robustness window's counters, and cell replay.

M4. Restricted to a surface

- `SurfaceDomain` over mesh-dev's `SurfaceMesh`. The `ClippedPolygon` (§5.6). The cell-centric traversal with its AABB tree (§7.4). The RDT builder, and the closed-ball checks.
- **Tests**: `rvd_surface` (§10.3 and §10.4): coverage, key census, nearest-seed sampling, the flat square against the box, the sphere and the torus, the thin plate.

- **You see** a surface tiled by coloured patches, its remesh, and the closed-ball violations lighting up wherever the sampling is too coarse. This is the milestone where the tool starts doing something no other one shows you.

M5. Integration and Lloyd

- The integrator with a density, Lloyd, then L-BFGS (§7.6). The energy plot, and stepping live.
- **Tests:** the gradient against finite differences, Lloyd’s monotone energy, and the second moment of the hexagonal lattice on a flat surface (§10.4), which checks the surface integrals.
- **You see** isotropic remeshing happen: a random sampling of a surface relaxing, frame by frame, into nearly hexagonal cells.

M6. Restricted to a volume

- `VolumeDomain` over mesh-dev’s `TetMesh`. The interior-cell shortcut through the winding number (§7.4). RDT tets. Viewer extension (b) of §8.8, if the carry-over has become annoying by then.
- **Tests:** a box as a tet mesh against the box itself, an L-shaped domain, a sphere, coverage.
- **You see** cells clipped by a curved volume, capped where the plane cuts them, and the Delaunay tets of the clipped diagram standing in the cut.

M7. Weights and transport

- Weights through every stage (the predicates have had them since M1). The weighted certificate. Newton on the weights.
- **Tests:** target masses reached to tolerance, the Hessian against finite differences, empty cells handled rather than fatal.
- **You see** cells trading volume until every one has its share, live.

M8. Speed and second opinions

- The element-centric traversal, the kd-tree, the Delaunay of §7.1.3, and an optional comparison against Geogram in the style of `bench_vs_geogram.cpp`.
- **Tests:** everything above, passed by the new code paths as well, and identical results between the paths.

M9. Outward

- Writers: `.obj` with a group per cell, `.msh` with the cell id as element data (so Gmsh shows it), and `.csv` of per-cell statistics. A Python module over pybind11, numpy in and numpy out, the way mesh-dev’s is.

10 Testing

10.1 The harness

mesh-dev’s own: one source file per suite, `check()` and `report_checks()` from `test_support.h`, quiet when passing, registered with `ctest`. Every random generator has a fixed seed. **Every diagram built in any test also runs every validator of §7.5**, so an invariant is checked hundreds of times on inputs nobody designed to break it, not once on a picked example.

10.2 Predicates against the oracle

- **Random inputs**, uniform in a unit box, and the same shifted by 10^6 so that the cancellation is severe. Every fast predicate must agree with the GMP oracle of §6.5 wherever the oracle is non-zero.
- **Near-degenerate inputs**, made from exact degenerate ones. Integer points on a sphere ($x^2 + y^2 + z^2 = r^2$ has many integer solutions) give five exactly cospherical seeds, and moving one coordinate by an ulp gives the hardest cases a filter can meet. Lattices and mirrored seed pairs give degenerate `side1` to `side3`, with domain vertices on bisectors.
- **Perturbation consistency** where the oracle says zero: `side1` is antisymmetric in its two seeds; the filtered and exact paths return the same perturbed sign; and the predicates that describe one vertex from different pieces agree about it.

10.3 Invariants, on everything

The validator table of §7.5 is the core of every diagram test: coverage, key census, neighbour symmetry, facet agreement, and nearest-seed sampling. They run on random seeds, on clustered seeds, on seeds outside the domain, and on every input of §10.4.

10.4 Known answers

The part that is fun to watch pass. Every row is a closed-form fact the engine must reproduce. The second moments are those of Conway and Sloane (1999), and G is defined in §7.5.

input	what must come out
cubic lattice, box aligned with the cells	every cell a cube: 6 faces of non-zero area, $G = 1/12$ to rounding. Every vertex needs §6.4
BCC lattice, interior cells	the truncated octahedron (Kelvin's cell): 14 faces (6 squares, 8 hexagons), 24 vertices, $G = 0.0785433$. Fully generic, so the counts are exact
FCC lattice, interior cells	the rhombic dodecahedron: 12 faces of non-zero area, $G = 0.0787451$. Six of its vertices are where four faces meet, so perturbation splits them: count after merging zero-length edges
uniform random seeds, cells away from the box	mean faces $\rightarrow 48\pi^2/35 + 2 \approx 15.54$, mean vertices $\rightarrow 96\pi^2/35 \approx 27.07$ (Meijering, 1953)
hexagonal lattice on a flat surface	regular hexagons, $G = 5/(36\sqrt{3}) \approx 0.0801875$: the surface path reproducing a 2D fact
seeds on a plane, a flat square as the domain	the surface RVD equals the 2D Voronoi diagram, which is also the box diagram of a thin slab around the plane, sliced: its cells are prisms
sphere, sampling fine enough	a valid RDT, $\chi = 2$. With n the seeds whose restricted cell is non-empty, the restricted neighbour counts (cells sharing an arc on the surface) sum to exactly $6n - 12$
torus, sampling fine enough	$\chi = 0$, and the restricted neighbour counts sum to exactly $6n$
seeds all on one sphere, in a box around it	every cell a cone from the centre, clipped by the box. The centre is a vertex of <i>all</i> n cells: the worst perturbation test there is
duplicate seeds	the later duplicate's cell is empty, and the count is reported
thin plate, a few seeds on one face only	every restricted cell reaches through to the other face: two components each, both found

10.5 Two ways, one answer

Every algorithm has a twin, and twins must agree.

pair	must agree on
brute force and meshless (§7.1)	the neighbour graph and the key sets, exactly; measures to 10^{-14} relative (the clipping order differs, so sums round differently)
cell-centric and element-centric (§7.4)	everything
a box domain, and the same box as a tet mesh	every cell's neighbours exactly, and its volume and centroid to rounding
filtered, and exact everywhere (§6.6)	everything, bit for bit
meshless and Delaunay (M8)	the neighbour graph inside B , exactly, degenerate inputs included

10.6 Optimisers

- The CVT gradient against central finite differences, at generic configurations, where the energy is smooth.
- Lloyd's energy never increases: with seeds free in space, and with seeds moved to the true nearest point of a surface (§3.4).
- L-BFGS reaches a lower energy than Lloyd in fewer passes.
- Transport: masses within tolerance, and the Laplacian Hessian against finite differences of the masses.

10.7 Against Geogram (optional)

A separate target, off unless pointed at a Geogram tree, exactly like `MESH_DEV_GEOGRAM_ROOT` in `mesh-dev`. I found no Geogram installation on this machine, so it would have to be built first. Same seeds, same surface: the cell areas agree to rounding, and on generic inputs the sets of RDT triangles are identical. On degenerate inputs the two perturbation schemes may legitimately differ, so only measures are compared there.

10.8 Determinism

The same diagram on 1, 2 and 8 threads, and twice in a row: one hash over every output array must be identical. This is the RVD version of `mesh-dev's bench_pipeline.py --compare` rule.

10.9 Benchmarks

Timings per stage, with the predicate decision paths printed next to them, so that a slowdown can be traced to the filter:

- the box: 10^4 , 10^5 and 10^6 seeds, uniform and clustered;
- a sphere of 10^4 triangles with 10^3 to 10^5 seeds, and a fine surface of 10^6 triangles with 10^4 seeds (both regimes of §7.4);
- a volume, with and without the interior-cell shortcut.

Timing on the laptop is noisy by up to a factor of two, as `mesh-dev's` notes say, so compare best-of- N runs, back to back.

10.10 The gallery

Every input of §10.4 is also a menu entry in the Diagram window. They are not automated tests, but they are where you look first when something is off, and the fastest way to show someone what the tool does.

11 From a Voronoi diagram to a restricted one

This is the question you asked, so here is the direct answer.

11.1 The short version

The plain Voronoi maker is stage A and stage B run on a domain with one piece, the box. The RVD adds domains with many pieces, a clipper for flat pieces, and a traversal that picks the pieces near each cell. It rewrites nothing. The predicates it needs with domain features (**side2** and **side3**) are already needed by the box, whose edges and faces are domain features too. So by the time the box works *exactly*, most of the RVD's difficulty is already solved.

concern	Voronoi in a box	RVD on a surface	RVD in a volume
domain pieces	1, the box	triangles	tets
clipper	ConvexCell from a box	ClippedPolygon from a triangle	ConvexCell from a tet
vertex types	V (corners), E (box edges), F (box faces), C	V, E, F	V, E, F, C
predicates	side1 to side4	side1 to side3	side1 to side4
traversal	none needed: one piece	cell-centric, with the AABB tree	the same, plus the interior-cell shortcut
welding keys	seeds and box features	seeds, mesh edges and triangles	seeds, mesh edges and faces
the dual	Delaunay tets whose Voronoi vertex lies in B	RDT triangles	RDT tets
integrals	volumes and moments	areas and moments	volumes and moments

11.2 The rules that keep the step small

These are what to hold on to while building the Voronoi part, because each one is cheap on day one and expensive to retrofit:

1. **The box is a domain from the first line.** Its corners, edges and faces are features with ids, and the vertices on them are symbolic V, E and F vertices. They are not floating-point intersections of six planes.
2. **Always stream (seed, piece, clipped piece),** even when there is only one piece. No consumer is ever allowed to assume that a cell is a single convex piece.
3. **Predicates talk about domain vertices,** never about planes computed from them. That keeps the degrees low, and it is what **side2** and **side3** will need on triangles and tets.
4. **Weights from day one,** zero by default.
5. **Stage A always clips the domain's bounding box B .** The restricted case needs exactly that, and nothing more.
6. **Write the surface arithmetic in dot products,** so that lifting to $\mathbb{R}^d$ later (§11.3) is a change of vector type, not of algorithm.

11.3 Further, in order of effort

generalisation	what changes	effort
power (Laguerre) diagrams	weights in f_{ij} , and the certificate uses the largest weight	already in the design

generalisation	what changes	effort
a density restricted to curves (boundary curves, feature lines)	linear ρ on every piece, closed-form integrals segments as pieces, a segment clipper, vertex types V and E, predicates <code>side1</code> and <code>side2</code>	already in the design small
labelled volumes (materials, inside and outside)	tets carry a label, restricted cells are split per (seed, label), and interfaces become facets of their own	small, and the bridge to mesh-dev's labelled domains
periodic box	the distance order and the bisectors use translated images of the seeds, and keys carry the image offset	moderate
anisotropic, on surfaces	lift the surface and the seeds to $\mathbb{R}^6$, as (x, sn) , the way the VorPaLine method of Lévy and Bonneel (2012) does. The surface clipper and <code>side1</code> to <code>side3</code> are dot products, so they work in any dimension	moderate, if rule 6 was kept
geodesic Voronoi on a surface	bisectors are no longer planes	a different project
Apollonius, or multiplicatively weighted	bisectors are hyperboloids or spheres	out of scope

11.4 Where it could meet your own work

These are possibilities for you to judge, not claims:

- **CVT remeshing of a target surface**, such as a bubble's interface, gives nearly regular triangles with density control, as an RDT. That is an alternative, or a complement, to remeshing it elsewhere.
- **A Voronoi diagram clipped by the interface** gives each cell's volume inside and outside the bubble: a volume fraction per cell, computed exactly rather than by sampling. You already have the winding numbers and the inside and outside labels.
- **Constant-volume Lagrangian cells** through transport (de Goes et al., 2015; Lévy, 2022) are a different discretisation of two-phase flow, and expensive to explore without exactly this kind of tool.
- **Point placement** for any mesh you seed, with the quality of every cell measured.

12 What makes this worth building

You said you cannot find good RVD software. The features below are the ones I have not seen in any of it. Each one is cheap *only because* of the decisions of §1: symbolic vertices, exact predicates with counters, streamed pieces, and a viewer you own.

12.1 A robustness microscope

Every vertex remembers how its existence was decided: by the filter, by exact arithmetic, or by perturbation. Two overlay channels show the last two, and the Robustness window counts all three per predicate. Look at random seeds and a handful of points light up. Look at a cubic lattice and every vertex does. Look at a symmetric part seeded symmetrically, and exactly its mirror planes light up, because that is where mesh vertices sit on bisectors. You see where your input is degenerate before it bites you somewhere else. Flip to *floating point only* and the validators show you what exact arithmetic was protecting: asymmetric neighbours, cracks in the coverage, a folded RDT, each with an address.

12.2 A closed-ball inspector

The closed ball property of §3.3 is the exact condition for the restricted Delaunay triangulation to be a valid mesh of the surface. The validators test every clause of it:

- each restricted cell is one disk;
- each pair of cells shares at most one arc;
- each Voronoi edge pierces the surface at most once.

Every violation becomes a point in a channel. That is precisely the map of where the sampling is too coarse for the surface’s features: thin parts, sharp creases, close sheets. It also suggests an experiment no existing tool makes easy. Insert a seed at each violation, at the point of the offending cell farthest from its seed, in the spirit of the refinement of Boissonnat and Oudot (2005). Relax with a few CVT steps and repeat until nothing lights up. The result is a CVT remesh that is valid by the theorem, not just by looking right.

12.3 Cell replay, and “why does this vertex exist?”

Select a cell, then step through its construction one bisector at a time with [and]. At each step the viewer shows the partial polytope, the bisector plane that cuts it next, the security sphere, and the distance to the next neighbour, and you watch the certificate fire. On a surface, the same works for one triangle being cut up by the cells that meet it. Select a vertex and the Cell window says what it is. For example: “F vertex on triangle 812, seeds 4, 17 and 23; side3 decided by the filter; constructed at (...)”. This is the debugger you would want on the day a result looks wrong, and it comes from stage A being local (§7.1.2).

12.4 Optimisation you can watch

Lloyd, L-BFGS and transport run a step per frame on moderate sizes, with the energy plotted live. On a surface you watch random cells relax into near-hexagons. In a box you watch Lloyd feel its way toward the truncated octahedron of the BCC lattice, which Gersho’s conjecture (Gersho, 1979) says is the best 3D cell, and which Du and Wang (2005) tested with CVTs. It is still a conjecture, and the tool will show you how close a given run gets, as G per cell. With transport, you watch the cells trade volume until every one has its target.

12.5 Any cell, instantly

Because a cell needs only its neighbours, the cell under the cursor of a ten-million-seed diagram can be computed on demand, without computing the other 9 999 999.

12.6 Results you can defend

The same answer on one thread and on sixty-four. Invariants checked on every diagram built. Constants reproduced that are known in closed form (§10.4). A second implementation for every first one (§10.5). When a number from this tool goes into a paper, you can say why it is right.

13 Decisions for you, and risks

13.1 Decisions

Each has a recommendation. None has to be settled before M0, but each is cheaper to settle before the milestone in the last column.

decision	options	recommendation	settle before
neighbour oracle first	meshless (§7.1.2), or Delaunay (§7.1.3)	meshless: it is the RVD’s own code, parallel and local. Delaunay later, as the second opinion	M3
the exact path	GMP rationals, or expansion arithmetic	GMP first, because it is obviously right. Expansions only if lattices get slow in practice	M1
mesh-dev’s code	reference it by path, copy it, or split out a mesh-core repository now	by path. Split later if both projects keep needing it	M0
viewer extension (b), topology revision	now, at M6, or never	at M6, when volumes make the id churn visible	M6
viewer extension (a), per-triangle labels	when the chromatic layers feel limiting, or never	when you want colours per cell, or labels on mesh-dev’s surfaces	any time
volume domains	require a tet mesh, or clip by a closed surface directly	a tet mesh. Gmsh makes them, and clipping by a non-convex surface is a robust boolean, a project of its own	M6
CVT on a surface	keep seeds on the surface (nearest point), or let them leave it	keep them on it, for remeshing, and keep the other as an option	M5
Python module	at M9, or earlier	as soon as a script needs cell volumes, since mesh-dev’s pattern makes it cheap	any time

13.2 Risks, and what contains them

risk	containment
a wrong error bound in a filter for side3 or side4	the oracle, on millions of cases, and exact everywhere having to give bit-identical results. If a static bound is hard to derive, start with an interval-arithmetic filter, which is slower but correct by construction
the exact path slow on degenerate input	acceptable for a research tool at first. The counters say exactly how often it runs, so there is no guessing
bad domain meshes: zero-area triangles, non-manifold scans	filtered when the domain is built, counted, and shown. Never passed to a predicate
memory beyond 10^7 seeds	the streamed consumers keep the core bounded. The display is the limit, so draw a region, not everything
viewer friction: id churn, composite order, one colour per layer	the workarounds of §8.4, then the two extensions of §8.8
scope creep	transport and anisotropy are tempting. The milestones put them after the surface and volume RVDs are solid, and every milestone has to end green

14 Glossary

term	meaning here
seed	one of the points p_i the diagram is built from, identified by its index
weight	w_i : zero for a Voronoi diagram, anything for a power (Laguerre) diagram
power distance	$\pi_i(x) = \ x - p_i\ ^2 - w_i$
bisector	the plane where $\pi_i = \pi_j$. One side of it is a half-space of cell i

term	meaning here
cell	V_i : the points closer to seed i , in power distance, than to any other seed
neighbour	a seed whose bisector supports a face of the cell
domain, piece	where the diagram is restricted to, as convex pieces: the box, triangles or tets
feature	a vertex, edge or face of the domain, with a global id
constraint	a plane bounding a clipped piece: a feature or a bisector
symbolic vertex	a vertex stored as what defines it: type V, E, F or C (§5.5)
key	a symbolic vertex's global identity: its feature and its seeds
predicate	a yes-or-no geometric question, answered exactly from input numbers
filter	the fast floating-point evaluation of a predicate, with a certified error bound
oracle	the slow, obviously correct reference implementation of a predicate
perturbation	Simulation of Simplicity: how exact ties are broken, consistently (§6.4)
certificate, security radius	the test that proves no farther seed can cut a cell (§7.1.2)
RDT	restricted Delaunay triangulation: the dual of the restricted cells
closed ball property	the condition under which the RDT is a valid mesh of the domain (§3.3)
CVT	centroidal Voronoi tessellation: every seed at its cell's centroid
G	a cell's dimensionless second moment: its quality, independent of its size
generation	one frozen result, published to the viewer as a whole (§5.9)
carrier	a mesh with vertices and no triangles, published so that channels can index its vertices
chromatic layer	one viewer layer per colour class of cells (§7.5, §8.2)

References

- N. Amenta and M. Bern. Surface reconstruction by Voronoi filtering. *Discrete & Computational Geometry*, 22(4):481–504, 1999.
- N. Amenta, S. Choi, and G. Rote. Incremental constructions con BRIO. In *Proceedings of the 19th Annual Symposium on Computational Geometry*, pages 211–219, 2003.
- G. Barill, N. Dickson, R. Schmidt, D. I. W. Levin, and A. Jacobson. Fast winding numbers for soups and clouds. *ACM Transactions on Graphics*, 37(4), 2018.
- J. Basselin, L. Alonso, N. Ray, D. Sokolov, S. Lefebvre, and B. Lévy. Restricted power diagrams on the GPU. *Computer Graphics Forum*, 40(2), 2021.
- J.-D. Boissonnat and S. Oudot. Provably good sampling and meshing of surfaces. *Graphical Models*, 67(5):405–451, 2005.
- J. H. Conway and N. J. A. Sloane. *Sphere Packings, Lattices and Groups*. Springer, 3rd edition, 1999.
- F. de Goes, K. Breeden, V. Ostromoukhov, and M. Desbrun. Blue noise through optimal transport. *ACM Transactions on Graphics*, 31(6), 2012.
- F. de Goes, C. Wallez, J. Huang, D. Pavlov, and M. Desbrun. Power particles: an incompressible fluid solver based on power diagrams. *ACM Transactions on Graphics*, 34(4), 2015.
- O. Devillers and M. Teillaud. Perturbations for Delaunay and weighted Delaunay 3D triangulations. *Computational Geometry: Theory and Applications*, 44(3):160–168, 2011.
- Q. Du, V. Faber, and M. Gunzburger. Centroidal Voronoi tessellations: applications and algorithms. *SIAM Review*, 41(4):637–676, 1999.
- Q. Du, M. Gunzburger, and L. Ju. Constrained centroidal Voronoi tessellations for surfaces. *SIAM Journal on Scientific Computing*, 24(5):1488–1506, 2003.

- Q. Du and D. Wang. The optimal centroidal Voronoi tessellations and the Gershgorin's conjecture in the three-dimensional space. *Computers & Mathematics with Applications*, 49(9–10):1355–1373, 2005.
- H. Edelsbrunner and E. P. Mücke. Simulation of simplicity: a technique to cope with degenerate cases in geometric algorithms. *ACM Transactions on Graphics*, 9(1):66–104, 1990.
- H. Edelsbrunner and N. R. Shah. Triangulating topological spaces. *International Journal of Computational Geometry and Applications*, 7(4):365–378, 1997.
- A. Gersho. Asymptotically optimal block quantization. *IEEE Transactions on Information Theory*, 25(4):373–380, 1979.
- J. Kitagawa, Q. Mérigot, and B. Thibert. Convergence of a Newton algorithm for semi-discrete optimal transport. *Journal of the European Mathematical Society*, 21(9):2603–2651, 2019.
- B. Lévy. A numerical algorithm for L_2 semi-discrete optimal transport in 3D. *ESAIM: Mathematical Modelling and Numerical Analysis*, 49(6):1693–1715, 2015.
- B. Lévy. Robustness and efficiency of geometric programs: the Predicate Construction Kit (PCK). *Computer-Aided Design*, 72:3–12, 2016.
- B. Lévy. Partial optimal transport for a constant-volume Lagrangian mesh with free boundaries. *Journal of Computational Physics*, 2022.
- B. Lévy and N. Bonneel. Variational anisotropic surface meshing with Voronoi parallel linear enumeration. In *Proceedings of the 21st International Meshing Roundtable*, 2012.
- Y. Liu, W. Wang, B. Lévy, F. Sun, D.-M. Yan, L. Lu, and C. Yang. On centroidal Voronoi tessellation: energy smoothness and fast computation. *ACM Transactions on Graphics*, 28(4), 2009.
- J. L. Meijering. Interface area, edge length, and number of vertices in crystal aggregates with random nucleation. *Philips Research Reports*, 8:270–290, 1953.
- A. Meyer and S. Pion. FPG: a code generator for fast and certified geometric predicates. In *Real Numbers and Computers (RNC 8)*, pages 47–60, 2008.
- N. Ray, D. Sokolov, S. Lefebvre, and B. Lévy. Meshless Voronoi on the GPU. *ACM Transactions on Graphics*, 37(6), 2018.
- C. H. Rycroft. Voro++: a three-dimensional Voronoi cell library in C++. *Chaos*, 19(4):041111, 2009.
- M. Sainlot, V. Nivoliers, and D. Attali. Restricting Voronoi diagrams to meshes using corner validation. *Computer Graphics Forum*, 36(5), 2017.
- J. R. Shewchuk. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry*, 18(3):305–363, 1997.
- D.-M. Yan, B. Lévy, Y. Liu, F. Sun, and W. Wang. Isotropic remeshing with fast and exact computation of restricted Voronoi diagram. *Computer Graphics Forum*, 28(5):1445–1454, 2009.
- D.-M. Yan, W. Wang, B. Lévy, and Y. Liu. Efficient computation of clipped Voronoi diagram for mesh generation. *Computer-Aided Design*, 45(4):843–852, 2013.